Chapter 5

그래프의 비밀

게임의 길찾기와 그래프 탐색

- 입력: 갈 수 있는 상태와 연결, 이동 비용
- 출력: 목적지까지의 노드 순서와 총비용
- 학습 흐름: 그래프 표현 → DFS·BFS → Dijkstra → A*
- 경로 계획과 실제 캐릭터 이동·충돌 회피는 서로 다른 단계

같은 맵도 그래프는 다를 수 있다

- 타일 격자: 셀을 노드로, 허용된 이웃 이동을 에지로
- 웨이포인트: 문·모퉁이·사다리 입구 같은 주요 지점을 노드로
- 노드가 적어져도 캐릭터가 통과할 수 없는 연결은 금지
- 낙하·점프·일방통행은 방향 에지로 표현

```
// 버그 수정
void Pathfinder::Render()
{
  gdi->TransparentText();
  if (m_pGraph == NULL) {     // m_pGraph가 NULL이 되는 경우가 있음
        return;
  }
```

원본 Pathfinder의 실제 탐색 조건

- 로컬 소스: VS2010/Buckland_Chapter5-Pathfinder
- 격자는 상하좌우뿐 아니라 대각선도 연결하는 8방향
- 기본 에지 비용은 노드 중심 사이의 실제 거리
- 지형 배수: 일반 1.0, 진흙 1.5, 물 2.0

그래프 클래스 구현하기

- GraphNode 클래스

```
class GraphNode {
protected:
  //every node has an index. A valid index is >= 0
  int        m_iIndex;
public:
  GraphNode():m_iIndex(invalid_node_index){}
  GraphNode(int idx):m_iIndex(idx){}
  GraphNode(std::ifstream& stream){char buffer[50]; stream >>buffer >>
    m_iIndex;}
  virtual ~GraphNode(){}
  int  Index()const{return m_iIndex;}
  void SetIndex(int NewIndex){m_iIndex = NewIndex;}
};
```

```
template <class extra_info = void*>
class NavGraphNode : public GraphNode {
protected:
  //the node's position
  Vector2D     m_vPosition;

  extra_info  m_ExtraInfo;
public:
  ...
};
```

GraphEdge 클래스

- 두 개의 그래프 노드 사이의 연결을 나타내기 위해 필요한 기본 정보를 캡슐화

```
class GraphEdge {
protected:
  //An edge connects two nodes. Valid node indices are >= 0.
  int     m_iFrom;
  int     m_iTo;
  //the cost of traversing the edge
  double  m_dCost;
public:
  GraphEdge(int from, int to, double cost):
     m_dCost(cost), m_iFrom(from), m_iTo(to) {}
  ...
};
```

노드의 번호·위치·상태

- Index: 배열에서 노드를 찾는 번호, 좌표 자체가 아님
- Position: 화면이나 월드에서 노드가 놓인 위치
- State: 앞으로 가능한 행동을 결정하는 정보 전체
- 같은 위치라도 열쇠·방향·연료가 다르면 다른 상태일 수 있음

실습 01 · 열쇠를 찾고 잠긴 문 통과하기

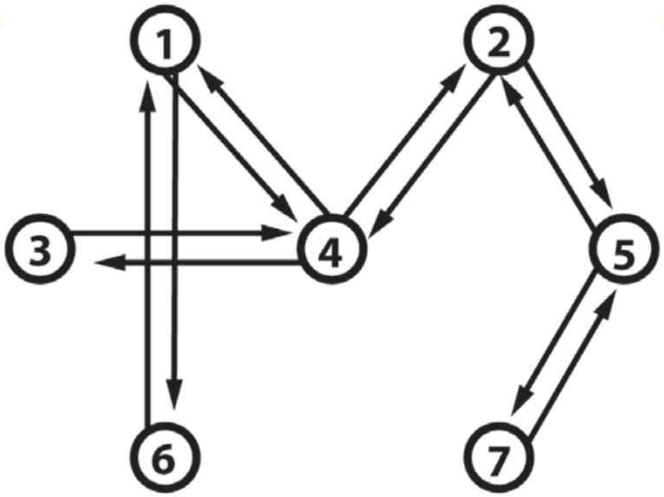
- 연결은 양방향, 이동 비용은 모두 1
- S—J—D—G, 그리고 J—K에서 K는 열쇠
- D 진입은 열쇠가 있을 때만 허용
- 미션: 상태 표현·최단 경로·위치만 방문할 때의 문제 설명

지점	S	J	K	D	G
역할	시작	분기	열쇠	잠긴 문	목표

SparseGraph 클래스

```
// 각 노드의 인덱스 숫자가 그래프 노드들의 벡터(m_Nodes)와 인접 에지 리스트
    (m_Edges)로 직접 연결
template <class node_type, class edge_type>
class SparseGraph {
public:
  typedef edge_type              EdgeType;
  typedef node_type              NodeType;
  //a couple more typedefs to save my fingers and to help with the formatting
  //of the code on the printed page
  typedef std::vector<node_type> NodeVector;
  typedef std::list<edge_type>       EdgeList;
  typedef std::vector<EdgeList>      EdgeListVector;
};
```

```
template <class node_type, class edge_type>
class SparseGraph {
public:
  typedef std::vector<node_type> NodeVector;
  typedef std::list<edge_type>       EdgeList;
  typedef std::vector<EdgeList>      EdgeListVector;
private:
  NodeVector      m_Nodes; // 이 그래프를 구성하는 노드들
  // 인접 에지 리스트의 벡터 (각 노드 인덱스는 그 노드와 관련된 에지의
  //                          리스트로 키(key)화 된다)
  EdgeListVector  m_Edges;
public:
  // invalid_node_index를 그 인덱스에 지정하여 노드를 제거
  void RemoveNode(int node);
};
```



	①	②	③	④	⑤	⑥	⑦
①	0	0	0	1	0	1	0
②	0	0	0	1	1	0	0
③	0	0	0	1	0	0	0
④	1	1	1	0	0	0	0
⑤	0	1	0	0	0	0	1
⑥	1	0	0	0	0	0	0
⑦	0	0	0	0	1	0	0

Figure 5.12. An adjacency matrix

① → 4 → 6
② → 4 → 5
③ → 4
④ → 1 → 2 → 3
⑤ → 2 → 7
⑥ → 1
⑦ → 5

Figure 5.13. An adjacency list representation of the digraph from Figure 5.6

인접 행렬과 인접 리스트 선택

- SparseGraph는 노드별 인접 에지 리스트를 유지
- 타일 맵은 대개 한 노드의 이웃 수가 작음
- 삭제된 번호와 배열의 크기, 활성 노드 수는 구분

항목	인접 행렬	인접 리스트
저장 공간	$O(V^2)$	$O(V + E)$
u→v 존재 검사	$O(1)$	$O(\deg(u))$
u의 이웃 순회	$O(V)$	$O(\deg(u))$

무정보 그래프 탐색

깊이 우선 탐색

```
template<class graph_type>
class Graph_SearchDFS {
  typedef typename graph_type::EdgeType Edge;
  typedef typename graph_type::NodeType Node;
private:
  //a reference to the graph to be searched
  const graph_type& m_Graph;
  std::vector<int> m_Visited; // 방문할때마다 unvisited ->visited 설정
  std::vector<int> m_Route;  // 각 노드의 부모를 기록하여 경로저장
  //this method performs the DFS search
  bool Search();
  ...
};
```

```
template <class graph_type>
bool Graph_SearchDFS<graph_type>::Search()
{
  //create a std stack of edges
  std::stack<const Edge*> stack;
  //create a dummy edge and put on the stack
  Edge Dummy(m_iSource, m_iSource, 0);
  stack.push(&Dummy);
  //while there are edges in the stack keep searching
  while (!stack.empty())
  {
    //grab the next edge
    const Edge* Next = stack.top();
    //remove the edge from the stack
    stack.pop();
    //make a note of the parent of the node this edge points to
    m_Route[Next->To()] = Next->From();
    //and mark it visited
    m_Visited[Next->To()] = visited;
```

```
    //if the target has been found the method can return success
    if (Next->To() == m_iTarget) {
       return true;    }
     //push the edges leading from the node this edge points to onto
     //the stack (provided the edge does not point to a previously
     //visited node)
     graph_type::ConstEdgeIterator ConstEdgeItr(m_Graph, Next->To());
     for (const Edge* pE=ConstEdgeItr.begin(); !ConstEdgeItr.end();
          pE=ConstEdgeItr.next())
     {
       if (m_Visited[pE->To()] == unvisited) {
         stack.push(pE);
       }
     }
   }
   //no path to target
   return false;
}
```



DFS의 방문 순서와 스택

- 이웃 방문 순서를 A→B로 하려면 스택에는 B→A로 넣음
- 스택은 마지막에 넣은 후보를 먼저 꺼내는 LIFO
- DFS는 경로를 찾지만 일반적으로 최단 경로를 보장하지 않음

단계	꺼낸 노드	스택: 아래 → 위
1	S	B, A
2	A	B, C
3	C	B, E
4	E	B, G
5	G	목표 발견, 종료

원본 DFS의 중복 후보 처리

- push 시 미방문이어도 같은 노드가 여러 번 대기할 수 있음
- pop 직후 이미 방문한 노드라면 건너뛰기
- 부모 기록은 중복 여부를 검사한 다음 수행

```
const Edge* next = stack.top();
stack.pop();
if (m_Visited[next->To()] == visited)
{
  continue;
}
m_Route[next->To()] = next->From();
m_Visited[next->To()] = visited;
```

실습

- DFS 개선하기
  - 반복적 깊이증가탐색 구현

반복적 깊이 증가 탐색의 재시작

- 깊이 제한을 0, 1, 2, …로 늘려 깊이 제한 DFS를 반복
- 제한마다 방문 기록·부모·깊이를 초기화
- Found / Cutoff / Failure를 구분하면 종료를 판단하기 쉬움
- 단위 비용에서 가장 얕은 목표를 찾으며, 비용 최적과는 구분

너비우선탐색

```
template <class graph_type>
bool Graph_SearchBFS<graph_type>::Search()
{
  //create a std queue of edges
  std::queue<const Edge*> Q;
  const Edge Dummy(m_iSource, m_iSource, 0);
  //create a dummy edge and put on the queue
  Q.push(&Dummy);
  //mark the source node as visited
  m_Visited[m_iSource] = visited;
  //while there are edges in the queue keep searching
  while (!Q.empty())
  {
    //grab the next edge
    const Edge* Next = Q.front();
    Q.pop();
    //mark the parent of this node
    m_Route[Next->To()] = Next->From();
```

```
    //exit if the target has been found
    if (Next->To() == m_iTarget) {
      return true;
    }
    //push the edges leading from the node at the end of this edge
    //onto the queue
    graph_type::ConstEdgeIterator ConstEdgeItr(m_Graph, Next->To());
    for (const Edge* pE=ConstEdgeItr.begin(); !ConstEdgeItr.end();
         pE=ConstEdgeItr.next()) {
      //if the node hasn't already been visited we can push the
      //edge onto the queue
      if (m_Visited[pE->To()] == unvisited) {
        Q.push(pE);
        //and mark it visited
        m_Visited[pE->To()] = visited;
      }
    }
  }
  //no path to target
  return false;
}
```

이 노드에서 나가는 edge들을 push

BFS는 큐에 넣을 때 방문 표시

- 먼저 들어온 후보를 먼저 꺼내는 FIFO
- 발견할 때 표시하면 같은 노드의 중복 대기를 방지
- 모든 에지 비용이 같을 때 최소 이동 횟수 경로를 보장

꺼낸 노드	꺼낸 뒤 큐: 앞 → 뒤	새로 발견
S	A, B	A, B
A	B, C	C
B	C, D	D
C	D, E	E
D	E, G	G
E	G	없음

실습 02 · 구조 드론의 최단 출동 경로

- 방향 연결: S→A,B / A→C / C→E / E→G
- 추가 연결: B→D / D→G, 이동 비용은 모두 1
- 이웃은 이름순, DFS 스택에는 역순으로 삽입
- 미션: DFS·BFS 방문 순서와 반환 경로 길이를 비교

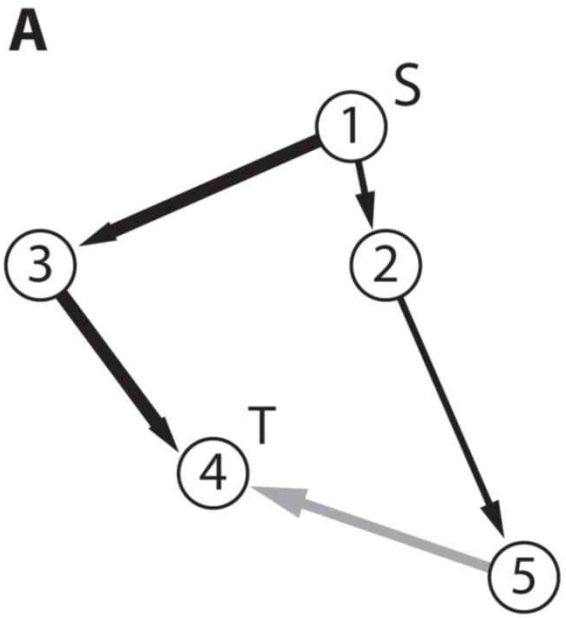
부모 배열로 경로 되돌리기

- parent[v]는 v 직전의 노드 번호
- 목표에서 부모를 따라 시작으로 돌아간 뒤 순서를 반전
- 도달 실패·시작=목표·삭제된 노드도 별도로 처리

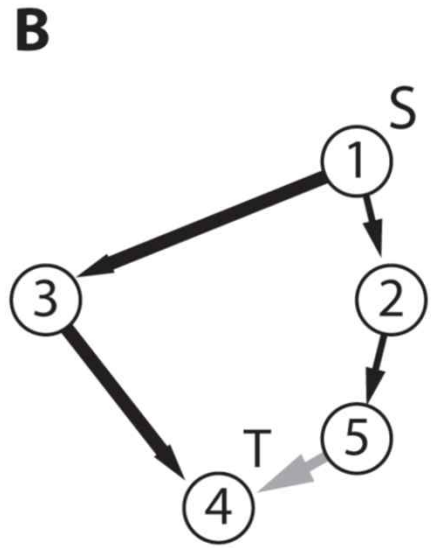
```
std::list<int> path;
if (!found)
{
  return path;
}
for (int node = goal; node != start; node = parent[node])
{
  path.push_front(node);
}
path.push_front(start);
```

비용 기반 그래프 탐색

- 에지 완화
  - BPSF: Best Path found So Far
  - 어떤 노드까지의 경로가 기존의 최상 경로 대신 새롭게 검사되는 에지를 사용함으로써 더 짧아질 것으로 추론되면, 그 에지가 추가되고 그 경로는 적절하게 갱신됨

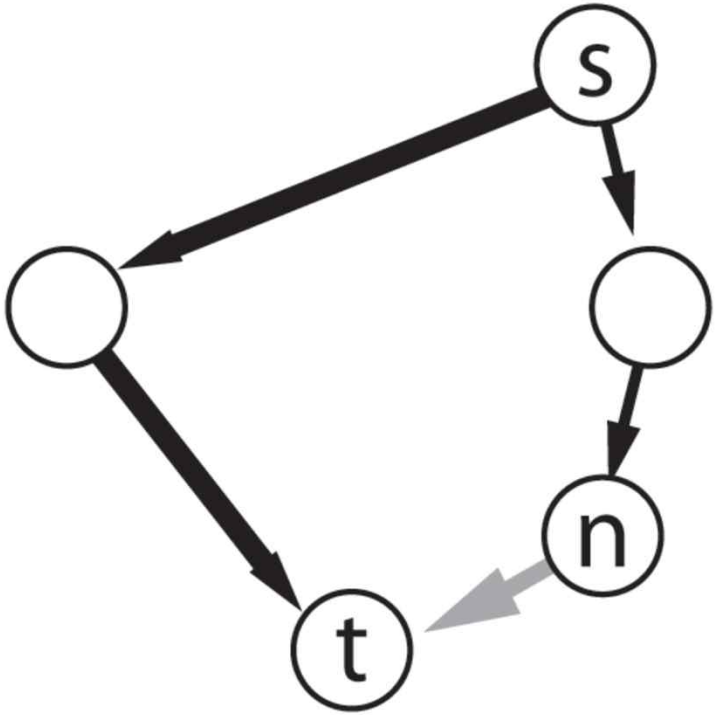


경로 갱신이 안되는 경우



경로가 갱신되는 경우

```
if (TotalCostToThisNode[t] > TotalCostToThisNode[n] + EdgeCost(n-to-t))
{
  TotalCostToThisNode[t] = TotalCostToThisNode[n] + EdgeCost(n-to-t);
  Parent(t) = n;
}
```



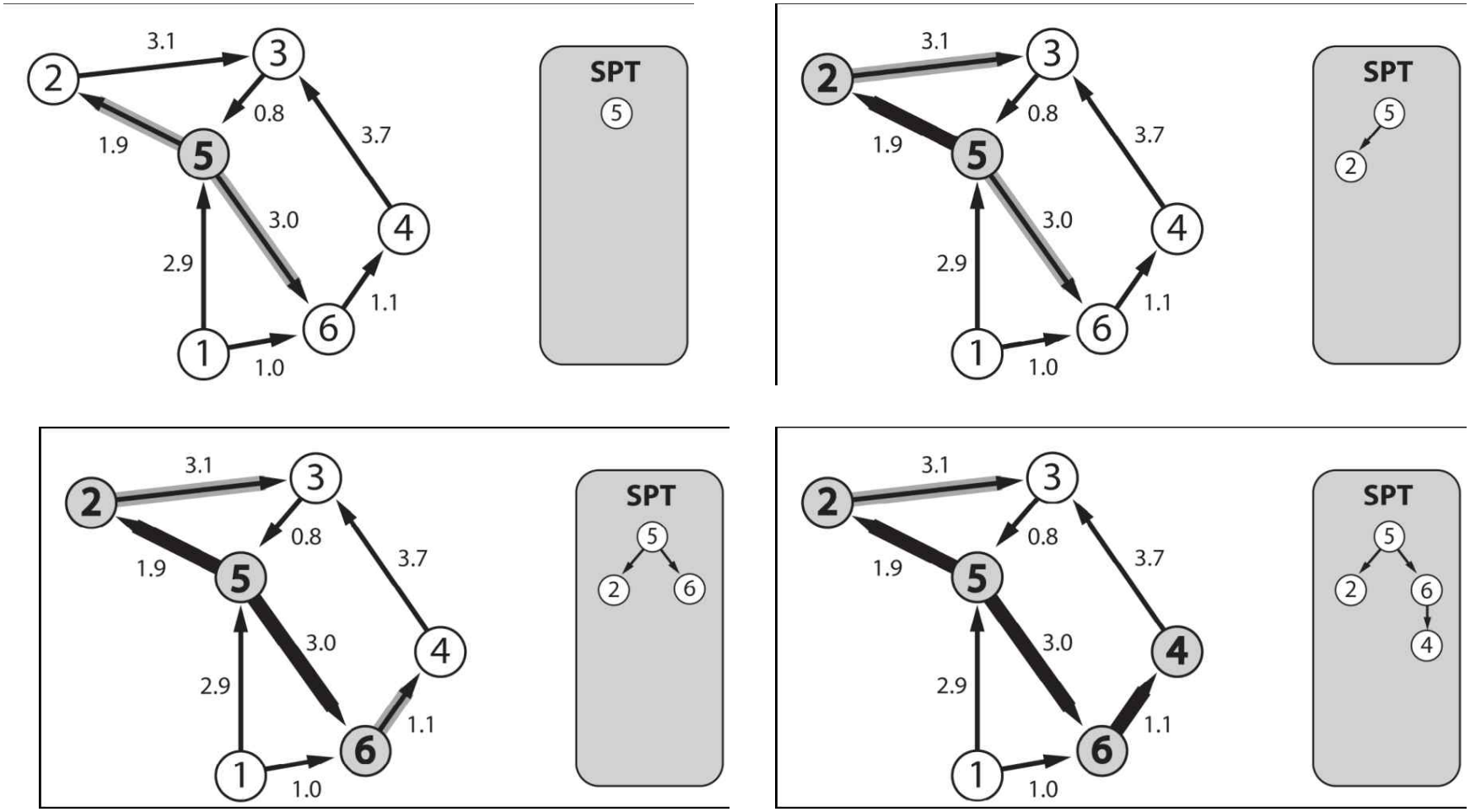
실습 03 · 늪지를 피해 보급품 배달하기

- 양방향 에지 비용은 아래 표와 같음
- BFS와 최소 비용 탐색의 반환 경로를 비교
- 미션: B의 비용·부모가 바뀌는 순간과 최종 비용 계산

에지	S—A	S—B	A—B	A—G	B—G
비용	2	5	1	9	2

Dijkstra 알고리즘

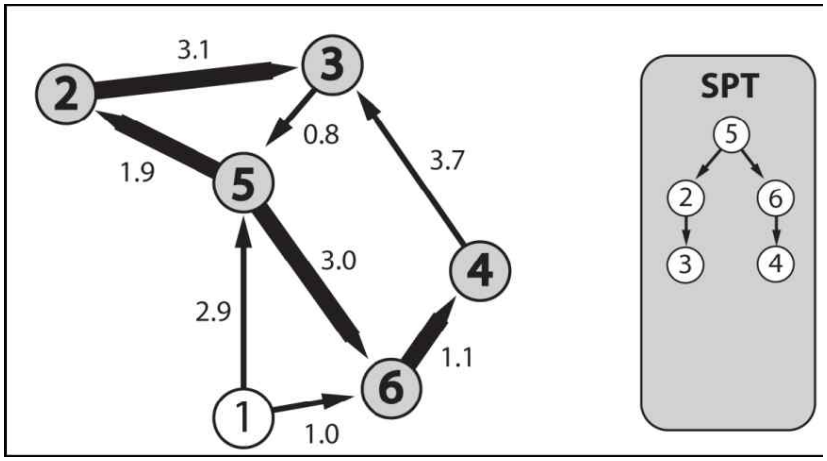
- Root 노드에서 시작하여 한번에 하나의 최단경로트리(SPT: Shortest Path Tree)에 없는 에지를 추가하여 최단경로트리를 구축



Dijkstra의 비용 확정 과정

- 최소 누적 비용 g의 미확정 노드를 꺼내 확정
- 전제: 에지 비용은 음수가 아님
- 목표를 후보로 넣을 때가 아니라 유효한 최소값으로 꺼낼 때 종료

꺼낸 노드	확정 g	주요 갱신	남은 후보
S	0	A=2, B=5	A2, B5
A	2	B=3, G=11	B3, G11
B	3	G=5	G5
G	5	목표 확정	종료



* 유의점은 frontier에 후보 edge를 넣는다

```
//this is indexed into by node index and holds the total cost of the best
//path found so far to the given node. For example, m_CostToThisNode[5]
//will hold the total cost of all the edges that comprise the best path
//to node 5, found so far in the search (if node 5 is present and has
//been visited)
std::vector<double>             m_CostToThisNode;

//this is an indexed (by node) vector of 'parent' edges leading to nodes
//connected to the SPT but that have not been added to the SPT yet. This is
//a little like the stack or queue used in BST and DST searches.
std::vector<const Edge*>     m_SearchFrontier;
```

```
template <class graph_type>
class Graph_SearchDijkstra
{
  typedef typename graph_type::EdgeType Edge;
  const graph_type&              m_Graph;
  //this vector contains the edges that comprise the shortest path tree -
  //a directed subtree of the graph that encapsulates the best paths from
  //every node on the SPT to the source node.
  std::vector<const Edge*>       m_ShortestPathTree;
  std::vector<double>             m_CostToThisNode;
  //this is an indexed (by node) vector of 'parent' edges leading to nodes
  //connected to the SPT but that have not been added to the SPT yet. This
  // is a little like the stack or queue used in BST and DST searches.
  std::vector<const Edge*>     m_SearchFrontier;//그 노드로의 부모edge 유지
```

```
template <class graph_type>
void Graph_SearchDijkstra<graph_type>::Search()
{
  //create an indexed priority queue that sorts smallest to largest
  //(front to back). Note that the maximum number of elements the iPQ
  //may contain is N. This is because no node can be represented on the
  //queue more than once.
  IndexedPriorityQLow<double> pq(m_CostToThisNode, m_Graph.NumNodes());

  //put the source node on the queue
  pq.insert(m_iSource);
```

```
//while the queue is not empty
while(!pq.empty())  {
  //get lowest cost node from the queue. Don't forget, the return value
  //is a *node index*, not the node itself. This node is the node not
  // already on the SPT that is the closest to the source node
  int NextClosestNode = pq.Pop(); // 최단 비용 순서는 pq에서 유지
  //move this edge from the frontier to the shortest path tree
  m_ShortestPathTree[NextClosestNode] = m_SearchFrontier[NextClosestNode];
  //if the target has been found exit
  if (NextClosestNode == m_iTarget) return;
  //now to relax the edges.
  graph_type::ConstEdgeIterator ConstEdgeItr(m_Graph, NextClosestNode);
  //for each edge connected to the next closest node
  for (const Edge* pE=ConstEdgeItr.begin();
       !ConstEdgeItr.end();
       pE=ConstEdgeItr.next())  {
    //the total cost to the node this edge points to is the cost to the
    //current node plus the cost of the edge connecting them.
    double NewCost = m_CostToThisNode[NextClosestNode] + pE->Cost();
```

그 노드로의 부모 edge임

그 노드에서 나가는 edge임

```
      //if this edge has never been on the frontier make a note of the cost
      //to get to the node it points to, then add the edge to the frontier
      //and the destination node to the PQ.
      if (m_SearchFrontier[pE->To()] == 0) {
        m_CostToThisNode[pE->To()] = NewCost;//소스부터 그 노드까지의 총비용
        pq.insert(pE->To());
        m_SearchFrontier[pE->To()] = pE;//NextClosestNode부터 이 노드로의
      }                                    //edge를 탐색 경계에 후보로 추가
      //else test to see if the cost to reach the destination node via the
      //current node is cheaper than the cheapest cost found so far. If
      //this path is cheaper, we assign the new cost to the destination
      //node, update its entry in the PQ to reflect the change and add the
      //edge to the frontier
      else if ( (NewCost < m_CostToThisNode[pE->To()]) &&
                (m_ShortestPathTree[pE->To()] == 0) ) {
        m_CostToThisNode[pE->To()] = NewCost;
        //because the cost is less than it was previously, the PQ must be
        //re-sorted to account for this.
        pq.ChangePriority(pE->To());
        m_SearchFrontier[pE->To()] = pE;
      }                    //pE->To()로의 기존 등록edge를 pE edge로 교체한다
    }
  }
}
```

아직 m_ShortestPathTree 에는 등록되지 않은 경우

후보·확정·미발견 상태의 구분

- frontier: 현재 가장 좋은 후보 부모 에지
- SPT: 확정된 노드의 부모 에지
- 출발 노드는 부모가 없으므로 nullptr만으로 미발견과 구분 불가
- 안전한 구현은 Unseen / Open / Closed 상태를 별도 기록

```
int u = pq.Pop();
state[u] = Closed;
for (const Edge* edge : outgoing(u))
{
  int v = edge->To();
  if (state[v] == Closed)
  {
    continue;
  }
  // unseen 여부와 더 낮은 g 여부를 검사
}
```

실습

- IndexedPriorityQLow<double>의 작동 방법을 설명하시오.

  ```
  void Graph_SearchDijkstra<graph_type>::Search()
       IndexedPriorityQLow<double>pq
  ```

  - 구현 내용 설명
  - 작동 방법 설명

IndexedPriorityQLow의 세 배열

- keys[node]: 그 노드의 비용 — g 또는 f
- heap[position]: 힙 위치에 들어 있는 노드 번호
- invHeap[node]: 그 노드가 현재 놓인 힙 위치
- 비용을 바꾼 뒤 ChangePriority(node)로 힙 순서를 복구

정보	예시	뜻
keys[3]	1	노드 3의 새 비용
heap[1]	3	힙 루트는 노드 3
invHeap[3]	1	노드 3이 루트에 있음

실습 04 · 업데이트된 보급 요청의 우선순위

- 노드 1·2·3·4의 비용을 7·3·9·5로 삽입
- 노드 3의 비용이 9에서 1로 줄어듦
- 미션: 필요한 호출과 이후 Pop 순서 설명
- 코드의 heap·invHeap이 서로 맞는지도 검사

특별한 Dijkstra: A*

Cost = AccumulativeCostTo(E.From) + E.Cost + CostTo(Target)

```
template <class graph_type, class heuristic>
class Graph_SearchAStar
{
private:
  //create a typedef for the edge type used by the graph
  typedef typename graph_type::EdgeType Edge;
private:
  const graph_type&               m_Graph;
  //indexed into my node. Contains the 'real' accumulative cost to that
node
  std::vector<double>              m_GCosts;
  //indexed into by node. Contains the cost from adding m_GCosts[n] to
  //the heuristic cost from n to the target node. This is the vector the
  //iPQ indexes into.
  std::vector<double>              m_FCosts;
  std::vector<const Edge*>        m_ShortestPathTree;
  std::vector<const Edge*>        m_SearchFrontier;//그 노드로의 부모edge
```

A*의 g·h·f는 서로 다른 값

- g(n): 시작에서 n까지 실제로 찾은 누적 비용
- h(n): n에서 목표까지 남은 최소 비용의 추정
- f(n) = g(n) + h(n): 후보를 꺼낼 우선순위
- h=0이면 Dijkstra, g를 무시하면 탐욕 최선 우선 탐색

후보	g	h	f=g+h
A	2	8	10
B	4	3	7 ← 먼저
C	6	4	10

Graph_SearchAStar

```
template <class graph_type, class heuristic>
class Graph_SearchAStar { ... };
class Heuristic_Euclid
{
public:
  Heuristic_Euclid(){}
  //calculate the straight line distance from node nd1 to node nd2
  template <class graph_type>
  static double Calculate(const graph_type& G, int nd1, int nd2)
  {
    return Vec2DDistance(G.GetNode(nd1).Pos(), G.GetNode(nd2).Pos());
  }
};
class Pathfinder {
  typedef SparseGraph<NavGraphNode<void*>, GraphEdge> NavGraph;...};
void Pathfinder::CreatePathAStar(){
  typedef Graph_SearchAStar<NavGraph, Heuristic_Euclid> AStarSearch;
  //create an instance of the A* search using the Euclidean heuristic
  AStarSearch AStar(*m_pGraph, m_iSourceCell, m_iTargetCell);
...}
```

이동 규칙에 맞는 휴리스틱

- 아래 식은 한 칸 길이 1, 지형 배수 최솟값 1 기준
- 좌표 차이 dx, dy는 절댓값
- 원본 화면 좌표를 사용하면 실제 에지 비용과 단위도 맞춤

이동 규칙	휴리스틱 h
4방향, 한 번에 1	dx + dy
8방향, 대각선 √2	max(dx,dy) + (√2−1)·min(dx,dy)
8방향, 모두 1	max(dx,dy)
거리 비용의 일반 연결	직선거리: 각 에지 비용 ≥ 실제 길이일 때

실습 05 · 순간이동 포털이 생긴 맵

- S=(0,0), G=(10,0), 포털 에지 S→G의 비용은 1
- 직선거리 휴리스틱 h(S)=10을 그대로 써도 될까?
- 미션: 허용 가능성과 안전한 대안을 설명

```
template <class graph_type, class heuristic>
void Graph_SearchAStar<graph_type, heuristic>::Search()
{ //create an indexed priority queue of nodes. The nodes with the
  //lowest overall F cost (G+H) are positioned at the front.
  IndexedPriorityQLow<double> pq(m_FCosts, m_Graph.NumNodes());
  //put the source node on the queue
  pq.insert(m_iSource);
  //while the queue is not empty
  while(!pq.empty()) {
    //get lowest cost node from the queue
    int NextClosestNode = pq.Pop();
    //move this node's parent edge from the frontier to the spanning tree
    m_ShortestPathTree[NextClosestNode] = m_SearchFrontier[NextClosestNode];
    //if the target has been found exit
    if (NextClosestNode == m_iTarget) return;
    //now to test all the edges attached to this node
    graph_type::ConstEdgeIterator ConstEdgeItr(m_Graph, NextClosestNode);
    for (const Edge* pE=ConstEdgeItr.begin(); !ConstEdgeItr.end();
         pE=ConstEdgeItr.next()) {
      //calculate the heuristic cost from this node to the target (H)
      double HCost = heuristic::Calculate(m_Graph, m_iTarget, pE->To());
```

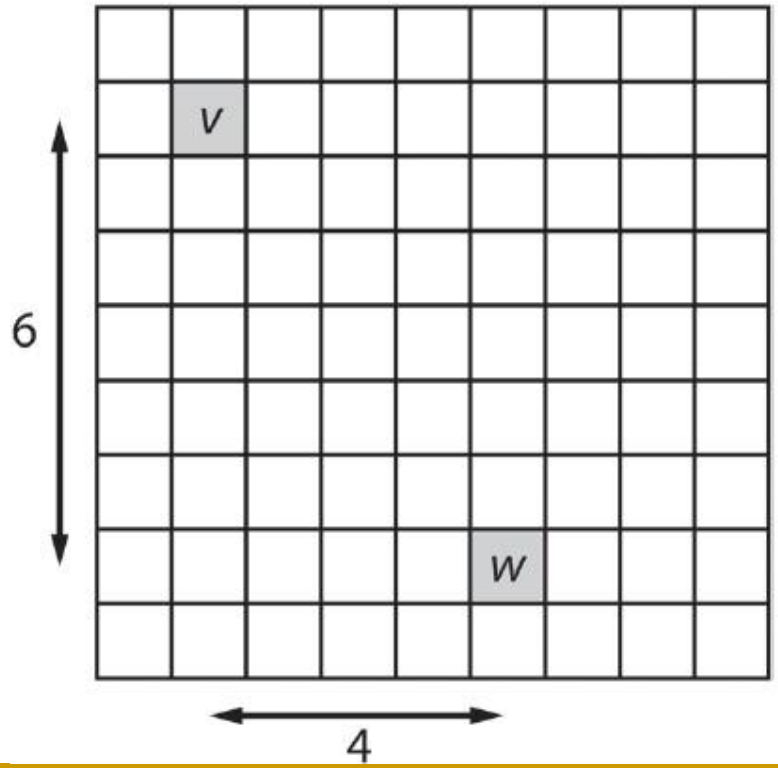
```
        //calculate the 'real' cost to this node from the source (G)
        double GCost = m_GCosts[NextClosestNode] + pE->Cost();
        //if the node has not been added to the frontier, add it and update
        //the G and F costs
        if (m_SearchFrontier[pE->To()] == NULL)  {
          m_FCosts[pE->To()] = GCost + HCost;
          m_GCosts[pE->To()] = GCost;
          pq.insert(pE->To());
          m_SearchFrontier[pE->To()] = pE;
        }
        //if this node is already on the frontier but the cost to get here
        //is cheaper than has been found previously, update the node
        //costs and frontier accordingly.
        else if ((GCost < m_GCosts[pE->To()]) && (m_ShortestPathTree[pE-
>To()]==NULL))  {
          m_FCosts[pE->To()] = GCost + HCost;
          m_GCosts[pE->To()] = GCost;
          pq.ChangePriority(pE->To());
          m_SearchFrontier[pE->To()] = pE; //그 노드로의 edge를 변경
        }
      }
    }
}
```

A*의 최적성과 닫힌 노드 재확장

- 허용 가능: $0 \le h(n) \le$ 실제 남은 최소 비용
- 일관성: 모든 에지에서 $h(u) \le c(u,v) + h(v)$, $h(G)=0$
- 원본처럼 Closed를 다시 열지 않는 구현은 일관성 확인
- 불일관하지만 허용 가능한 h라면 개선된 Closed 노드를 재개방

A*

- 휴리스틱 class를 맨하탄 거리로 변환
  - 맨하탄 거리: 가로와 세로 방향의 변이의 합
  - Euclidean 거리와의 성능(시간) 비교
- 휴리스틱을 Euclidean 거리 제곱으로 변환하고 시간 비교



거리 제곱은 같은 A*가 아니다

- 제곱근을 없애면 h의 크기와 비용 단위도 달라짐
- S=(0,0), A=(1,0), G=(4,0)
- 에지 비용: S→A=1, A→G=3, S→G=5
- h(A)=3이면 f(A)=4, h^2(A)=9이면 f(A)=10

h 선택	S 확장 후 A의 f	G의 f	반환 비용
유클리드	1 + 3 = 4	5	4 (최적)
거리 제곱	1 + 9 = 10	5	5 (비최적)

휴리스틱 비교 실험의 기록

- 같은 맵·출발·목표·이웃 순서·동률 규칙을 사용
- Dijkstra의 결과 비용을 정답 기준으로 삼음
- 최종 비용·확장 수·큐 최대 크기·시간을 함께 기록
- Release, 준비 실행 후 여러 회 측정의 중앙값 비교

방법	성공?	경로 비용	확장 노드	시간 중앙값
Dijkstra	기록	기준값	기록	기록
A* + 안전한 h	기록	기준과 비교	기록	기록
A* + 거리 제곱	기록	불일치도 기록	기록	기록

실습 06 · 보스방 길찾기 성능 경주

- 4방향·단위 비용 맵에서 h=0 / Euclid / Manhattan 비교
- 별도 8방향 맵에서는 Euclid / Octile 비교
- 거리 제곱은 최적성 위반을 찾는 별도 실험
- 미션: 빠르기뿐 아니라 정답 비용을 유지하는지도 보고

N 퀸(queen) 퍼즐

- N×N 체스 판 위에 N개의 퀸을 서로 잡지 못하도록 배치하는 문제 (각 행·열에 정확히 하나, 각 대각선에 최대 하나의 퀸을 둔다).
  - 시작 노드는 빈 N×N 배열
  - 후속자 함수는 하나의 퀸을 정당한 임의의 위치에 추가한 새로운 N×N 배열을 만들어낸다
  - 목표 술어는 배열 안에(정당한 위치에) N개의 퀸이 존재하는 경우에만 만족

N-퀸: 상태 공간 탐색과 배치 개선

- 제약: 각 행·열에 정확히 1개, 각 대각선에는 최대 1개
- 구성 탐색: 빈 배치에서 열마다 안전한 퀸을 하나씩 추가
- 지역 탐색: 모든 열에 퀸을 놓고 충돌 수를 줄임
- 두 방법은 상태·후속자·평가 함수가 다름

실습

- N 퀸 퍼즐에 대한 해
  - $\hat{h}(n) \geq 0$, A* 그래프 탐색으로 풀이
  - 언덕 오르기(Hill-climbing)
  - 무작위 재시작 언덕 오르기(Random restart)
    - 옵션: 횡이동 (sideway move)

충돌 수와 무작위 재시작

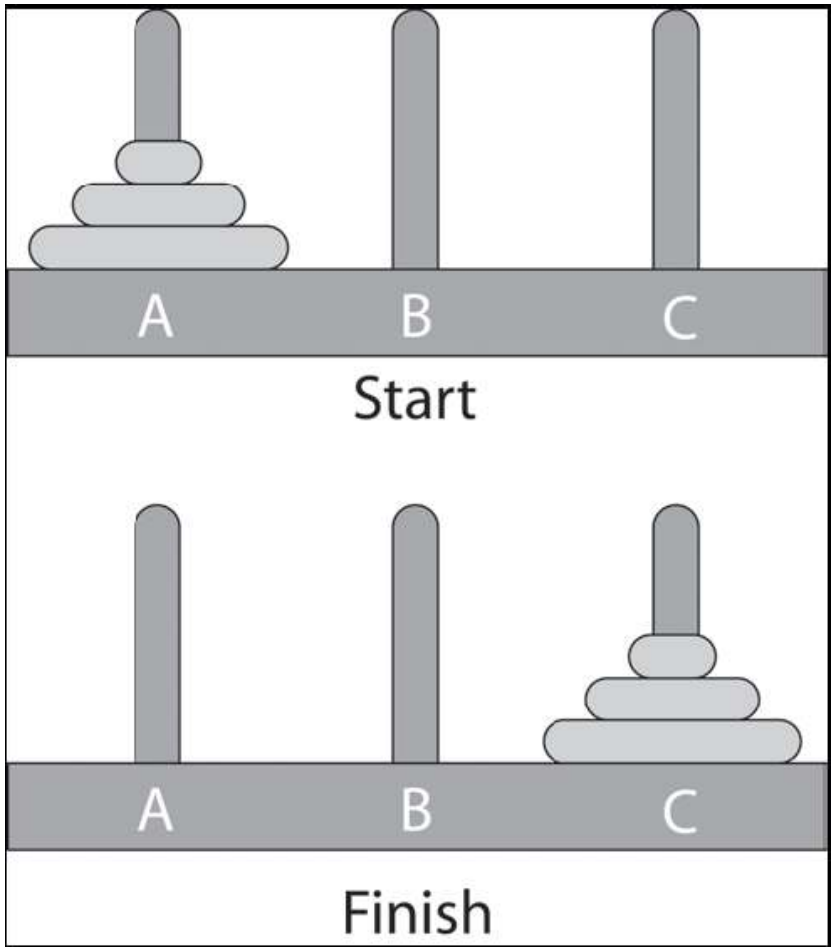
- 지역 탐색의 평가값 C: 서로 공격하는 퀸 쌍의 수
- C가 줄어드는 이웃으로 이동, C=0이면 성공
- 개선이 없으면 제한된 횡이동 또는 새 배치로 재시작
- A*의 남은 비용 h와 충돌 평가값 C를 같은 것으로 보지 않음

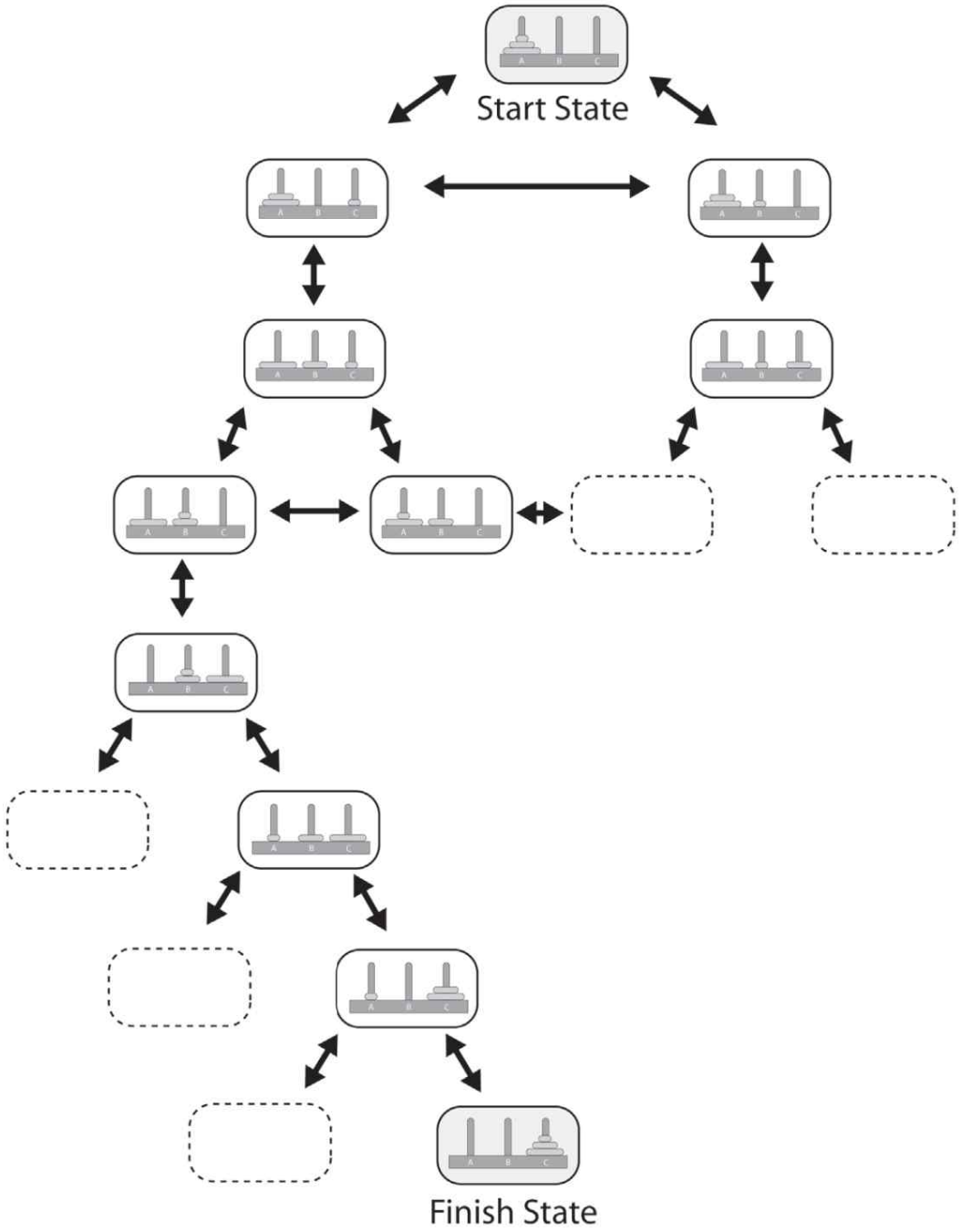
실습 07 · 레이저 보안장치의 안전 배치

- 4열 배열 q[c]=해당 열 퀸의 행 번호, 행은 0부터 시작
- 시작 q=[0,0,0,0], 변경 q=[0,3,0,0]
- 미션: 공격하는 쌍 수를 계산하고 안전 배치 하나 제시
- 확장: 8-퀸에서 재시작·횡이동 제한별 성공률 비교

배치	충돌 쌍 수: 학생 계산
[0, 0, 0, 0]	?
[0, 3, 0, 0]	?
직접 만든 안전 배치	0인지 검증

n 디스크 하노이 탑




Start State
Finish State

하노이 탑의 상태와 후속자

- 상태 p[d]: 작은 디스크부터 각각 놓인 기둥 번호
- 각 기둥의 맨 위 디스크만 다른 기둥으로 이동
- 큰 디스크를 작은 디스크 위에 놓는 이동은 금지
- 같은 상태 재방문을 막지 않으면 왕복 이동이 반복됨

실습

- n 디스크 하노이 탑 퍼즐에 대한 해
  - $\hat{h}(n) \geq 0$, A* 그래프 탐색으로 풀이

실습 08 · 마법 디스크 운반 퍼즐

- 3개 디스크를 A에서 C로 옮기는 합법 최단 경로 탐색
- BFS와 A*: h=목표 기둥 밖의 디스크 수 비교
- 미션: 부모로 경로를 복원하고 모든 이동을 검증
- 확장: n=1~6의 비용을 2^n-1과 비교

게임 루프 안의 길찾기

- 실시간 이동: 경로 계획 → 다음 목표점 → Steering·충돌 처리
- 여러 프레임에 탐색을 나누면 Open·g·parent를 유지
- 문이 닫히거나 지형이 바뀌면 경로와 탐색 결과를 재검증
- 다수 개체가 같은 목표로 가면 목표 기준 거리장을 재사용

실습 09 · 좀비 100마리의 출구 찾기

- 5×5, 4방향·단위 비용, ■는 통과 불가
- S1·S2·S3에서 G까지의 거리와 첫 이동 방향 계산
- 미션: G에서 한 번 탐색한 거리장을 모든 좀비가 공유
- 확장: (2,2)의 문이 열리면 어느 값부터 달라질까?

y / x	0	1	2	3	4
0	S1	·	■	·	·
1	·	·	■	·	·
2	S2	·	■	·	G
3	·	S3	■	·	·
4	·	·	·	·	·

알고리즘 선택과 검증 기준

- 자료구조보다 먼저 상태·허용 이동·비용을 정의
- 최단이라는 말은 어떤 비용 기준인지 함께 명시
- 방문 시점·비용 개선·종료 조건을 코드와 연결

목적·조건	기본 선택	정답 확인
도달 가능·깊게 탐색	DFS	연결·부모 유효성
동일 비용 최단 경로	BFS	최소 에지 수
비음수 가중 비용	Dijkstra	유효 최소 g에서 확정
목표 지향 + 검증된 h	A*	기준 비용·h 조건
좋은 최종 배치	지역 탐색·CSP	모든 제약 만족